

Национальный исследовательский университет
"Высшая школа экономики"

Исследовательский проект
Обучение ретривера на инструкциях
Promptriever: Instruction-Trained Retrievers

Выполнил:

Студент группы БПМИ242
Хасанов Айдар Рифкатович

Научный руководитель:

Андреева Дарья Александровна

Москва, 2026

Содержание

Аннотация	ii
1 Введение	1
1.1 Актуальность	1
1.2 Цели	1
1.3 Задачи проекта	1
2 Обзор и анализ источников	3
2.1 Promptriever	3
2.2 InstructOR	3
2.3 Multilingual-E5	4
2.4 BGE-M3	4
2.5 SberQuAD	5
2.6 Метрики оценки	5
2.6.1 DCG и nDCG	5
2.6.2 MRR и pMRR	5
2.7 mFollowIR	6
2.8 ruMTEB	6
3 Основная часть	8
3.1 Первые эксперименты	8
3.2 Сбор тренировочного датасета	8
3.3 Дообучение модели	10
3.4 Latency	12
4 Заключение	13
5 Использование ИИ	14
6 Дальнейшая работа	15
Список использованных источников	16

Аннотация

Данная работа посвящена реализации подхода Promptriever для русского языка путем дообучения нескольких моделей на модифицированном датасете SberQuAD. Целью исследования является создание поисковой системы, способной эффективно следовать текстовым инструкциям пользователя для повышения точности и управляемости поиска. Эффективность метода оценивается на бенчмарках ruMTEB и mFollowIR с использованием метрик nDCG@10 и pMRR.

1 Введение

Поиск информации прошел путь от простого сопоставления ключевых слов до использования семантических векторов, создаваемых нейронными сетями. Современные би-энкодеры позволяют находить документы по смыслу, переводя запросы и документы в единое векторное пространство $\mathbb{R}^d$. Однако традиционные модели поиска ограничены: они не способны учитывать специфические инструкции пользователя, воспринимая их как часть поискового запроса.

Данная работа посвящена реализации подхода, использованного в статье Promptriever [8] для русского языка. Основная идея заключается в обучении модели воспринимать текстовые инструкции.

1.1 Актуальность

В последние годы методы информационного поиска претерпели значительную трансформацию благодаря внедрению плотных векторных представлений на базе предобученных языковых моделей. Несмотря на высокую эффективность в классических задачах поиска по сходству, современные ретриверы могут игнорировать специфические инструкции пользователя.

В реальных сценариях запросы часто содержат не только описание темы, но и дополнительные ограничения. Обычные би-энкодеры, такие как multilingual-e5, не способны отличить релевантный документ от документа, который близок по теме, но нарушает заданную инструкцию.

Актуальность данной работы обусловлена необходимостью разработки и имплементации подходов, которые позволяют адаптировать би-энкодеры к следованию инструкциям. Это критически важно для создания умных ассистентов и RAG-систем, где точность следования промпту напрямую определяет качество итогового ответа системы.

1.2 Цели

1. Изучение и реализация модели Promptriever, которая умеет работать с инструкциями на естественном языке при выполнении поиска документов.
2. Оценка эффективности модели в различных условиях.
3. Сравнение различных моделей в качестве baseline при реализации ретривера.

1.3 Задачи проекта

1. Изучение архитектуры, предложенной в статье Promptriever.

2. Подготовка обучающего датасета на основе SberQuAD [3] с генерацией позитивных и негативных инструкций для обучения модели следованию командам.
3. Дообучение multilingual-e5-base [6] и других би-энкодеров.
4. Оценка качества базового поиска на бенчмарках ruMTEB [4] по метрике nDCG@10.
5. Оценка способности модели следовать инструкциям на retrieval-бенчмарке mFollowIR [7] с использованием метрики pMRR.
6. Сравнение разных моделей в качестве baseline.

2 Обзор и анализ источников

В основу исследования легли следующие работы:

2.1 Promptriever

Статья Promptriever: Instruction-Trained Retrievers Can Be Prompted Like Language Models [8] является основной теоретической и методологической опорой данной работы. В ней вводится подход к обучению ретриверов, при котором модель должна учитывать не только семантическую близость запроса и документа, но и дополнительные условия, заданные в текстовой инструкции. Это отличает Promptriever от классических би-энкодеров: обычная модель поиска стремится приблизить вектор запроса к вектору тематически релевантного документа, тогда как Promptriever дополнительно учится различать документы, которые подходят по теме, но нарушают ограничение из промпта.

Promptriever обучается на инструкциях разного вида, поэтому может использоваться в zero-shot сценариях, где заранее неизвестны ни формулировка пользовательского запроса, ни тип ограничения. Особую роль в обучении играют негативные примеры: авторы используют документы, которые семантически близки к запросу, но не удовлетворяют инструкции. Такая постановка заставляет модель обращать внимание на значимые слова в условии и приводит к заметному росту метрики pMRR: в статье сообщается о приросте на +14.3 по сравнению с RepLLaMA. Кроме того, авторы показывают масштабируемость подхода: качество следования инструкциям повышается при увеличении размера базовой модели и объема синтетических данных.

2.2 InstructOR

Работа One Embedder, Any Task: Instruction-Finetuned Text Embeddings [5] также рассматривает идею использования инструкций при построении эмбеддингов. InstructOR обучается переводить в векторное пространство не только исходный текст, но и описание задачи, что делает одну и ту же модель применимой к разным сценариям.

Однако между InstructOR и Promptriever есть существенное различие. InstructOR обучался на фиксированных инструкциях вида `represent the {domain} for {task type} supporting documents; input: {input}`, тогда как Promptriever может работать с произвольными инструкциями (впрочем, это зависит от модели, которую дообучают). Кроме того, Promptriever использует при дообучении негативные инструкции к запросам, благодаря чему модель учится отличать документы, которые релевантны теме, от документов, которые одновременно релевантны теме и удовлетворяют условию. Это отражается и в качестве: в статье о Promptriever средний

pMRR на FollowIR составляет +11.2 против -2.8 у Instructor XL, а на MS-MARCO значение nDCG у Promptriever также существенно выше: 92.1 против 48.6 [8].

При этом подход InstructOR хорошо обобщается на широкий набор задач, включая классификацию, retrieval и оценку генерации текста. Также инструкции в InstructOR были собраны вручную, поэтому он менее подвержен шуму, который может возникать при синтетической генерации инструкций.

2.3 Multilingual-E5

Технический отчет Multilingual E5 Text Embeddings [6] описывает модель multilingual-e5-base, которая используется в данной работе в качестве одной из базовых моделей для дообучения. Для настоящего исследования она особенно важна, поскольку изначально обучалась как мультиязычная модель и поддерживает русский язык.

Обучение multilingual-e5-base состоит из нескольких этапов. Сначала модель проходит контрастивное предобучение на 10^9 парах вида текст-текст из открытых источников, а затем дообучается на $1.6 \cdot 10^6$ текстах и $5 \cdot 10^5$ инструкциях, сгенерированных GPT-3.5/4. При этом запросы и документы подаются в модель как `query: {query}` и `passage: {passage}`. Это необходимо учитывать при подготовке обучающего датасета и при последующем тестировании, поскольку несоблюдение формата может ухудшить качество эмбедингов. Размер модели составляет 278М параметров, что делает ее достаточно удобной для экспериментов, но при этом достаточно сильным baseline решением.

2.4 BGE-M3

В статье M3-Embedding [2] была представлена модель bge-m3. Она мультиязычна, также поддерживает русский язык, поэтому подходит для сравнения с multilingual-e5-base. В отличие от моделей, ориентированных только на dense retrieval, bge-m3 объединяет несколько режимов поиска: dense retrieval, sparse retrieval и multi-vector retrieval.

В отличие от multilingual-e5-base, bge-m3 не требует обязательного форматирования входа в виде `query: {query}`, что упрощает подготовку данных и снижает риск ошибки при инференсе. Кроме того, она поддерживает контекст до 8192 токенов, поэтому может применяться для поиска по длинным документам без предварительного разбиения на короткие фрагменты. При этом bge-m3 крупнее multilingual-e5-base: ее размер составляет 569М параметров. В работе она используется как сильный baseline, а также проводятся эксперименты по ее дообучению.

2.5 SberQuAD

Работа SberQuAD: Russian Reading Comprehension Dataset: Description and Analysis [3] посвящена созданию и анализу первого крупного русскоязычного датасета для задачи смыслового чтения. SberQuAD является русскоязычным аналогом SQuAD и содержит более $5 \cdot 10^4$ троек вида “контекст, вопрос, ответ”, собранных с помощью краудсорсинга на базе статей Википедии.

В данном исследовании SberQuAD используется не как датасет для question answering в классической постановке, а как основа для построения обучающих пар для ретривера. Наличие контекста и связанного с ним вопроса позволяет естественным образом сформировать задачу поиска релевантного фрагмента по запросу. Далее такие пары можно модифицировать с помощью позитивных и негативных инструкций, превращая исходный QA-датасет в материал для обучения модели следованию пользовательским ограничениям.

2.6 Метрики оценки

Для оценки качества поиска в работе используются две метрики: nDCG@10 и pMRR. Первая отражает качество обычного ретрива, а вторая учитывает, насколько хорошо модель следует инструкциям к запросу.

2.6.1 DCG и nDCG

DCG@10 используется для оценки ранжирования релевантных документов в первых десяти позициях выдачи. Сначала считается Discounted Cumulative Gain:

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}$$

где rel_i – релевантность документа на позиции i . Чем выше расположен релевантный документ, тем больший вклад он дает в итоговый скор. Затем значение нормируется на идеальное ранжирование:

$$\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

где $\text{IDCG}@k$ – максимальное возможное значение $\text{DCG}@k$ для данного запроса.

2.6.2 MRR и pMRR

Для одного запроса reciprocal rank оценивает позицию первого релевантного документа:

$$\text{RR} = \frac{1}{\text{rank}}$$

где rank – позиция первого правильного документа в выдаче. Тогда MRR считается как среднее значение RR по всем запросам:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

В mFollowIR используется модификация этой идеи – pMRR . Пусть R_{og} – ранг документа при исходном запросе, R_{new} – ранг документа при новом запросе. Для каждого документа, у которого изменилась релевантность, считается:

$$\text{pMRR}(d) = \begin{cases} \frac{R_{\text{new}}}{R_{\text{og}}} - 1, & \text{если } R_{\text{og}} > R_{\text{new}}, \\ 1 - \frac{R_{\text{og}}}{R_{\text{new}}}, & \text{иначе.} \end{cases}$$

Интуиция следующая: если ранги не поменялись, то инструкция ни на что не повлияла и $\text{pMRR} = 0$. Если документ, который должен был стать менее релевантным после изменения инструкции, поднялся выше, то $\text{pMRR} < 0$, а если опустился ниже, то $\text{pMRR} > 0$.

2.7 mFollowIR

Статья mFollowIR [7] вводит мультязычный бенчмарк для оценки того, насколько хорошо ретриверы следуют инструкциям. Для данной работы этот источник особенно важен, потому что mFollowIR включает русский язык и позволяет измерять не только тематическую релевантность выдачи, но и выполнение дополнительных условий, заданных пользователем.

Бенчмарк содержит запросы с ограничениями разного типа: например, по временному интервалу, стилю текста или подмножеству сущностей. Такая постановка ближе к реальным поисковым сценариям, где пользователь часто ищет не просто документы по теме, а документы, удовлетворяющие конкретному условию. Для оценки используется метрика pMRR , которая штрафует модель, если в верхние позиции выдачи попадает документ, релевантный теме, но нарушающий инструкцию. Поэтому ситуация, в которой модель имеет высокий nDCG и низкий pMRR , означает, что она умеет находить тематически близкие документы, но плохо следует инструкциям.

2.8 ruMTEB

Работа The Russian-focused embedders’ exploration: ruMTEB benchmark and Russian embedding model design [4] представляет ruMTEB — русскоязычное расширение глобального бенчмарка MTEB. Авторы объединяют 23 датасета, разделенных на 7 категорий задач, включая классификацию, кластеризацию и поиск. Благодаря этому ruMTEB позволяет оценивать embedding-модели не на одной узкой задаче, а на широком наборе сценариев, характерных для русского

языка.

В рамках данной работы ruMTEB используется прежде всего для оценки базового качества поиска. Для retrieval-задач бенчмарк включает такие наборы данных, как RuBQ, RiaNews и русскоязычная версия MIRACL. Эти датасеты позволяют проверить, сохраняет ли модель качество обычного семантического поиска после дообучения на инструкциях. Такое разделение оценивания важно: mFollowIR показывает способность модели следовать условиям, а ruMTEB помогает убедиться, что улучшение instruction following не достигается ценой деградации стандартного retrieval-качества.

3 Основная часть

3.1 Первые эксперименты

На начальном этапе работы была проведена оценка моделей multilingual-e5-base и bge-m3 без дообучения на инструкциях. Тестирование проводилось на датасете RuBQRetrieval (часть бенчмарка ruMTEB) и mFollowIR.

Таблица 1: Результаты тестирования без дообучения

Модель	RuBQRetrieval (nDCG@10)	mFollowIR (pMRR)
multilingual-e5-base ¹	69.170	−2.994
bge-m3	71.178	−2.154

¹ Для multilingual-e5-base использовался формат `query: {запрос}` и `passage: {текст}`

Как видно из таблицы, без дообучения модели показывают неплохие результаты по метрике nDCG, но достаточно слабые результаты по метрике pMRR. Дальнейшие эксперименты будут направлены на обучение модели таким образом, чтобы при сохранении высокого качества поиска она демонстрировала значительный прирост по метрике pMRR.

3.2 Сбор тренировочного датасета

За основу датасета для дообучения на инструктивных негативах берется SberQuAD. Для каждого примера (контекст, вопрос, ответ) генерируются позитивные и негативные инструкции. Позитивные инструкции формулируются так, чтобы модель должна была выдавать релевантные документы, а негативные инструкции содержат условия, которые нарушают релевантность.

Инструкции генерировались синтетически с помощью LLM. В качестве генератора использовалась Llama3-8b [1].

Разберем генерацию датасета по шагам. По паре (`query`, `context`), где `context` – отрывок из текста (взятый из исходного датасета SberQuAD), `query` – вопрос (также взятый из SberQuAD), строилась пара (`query` + `instruction`, `context`), `instruction` – сгенерированная инструкция.

Промпт для генерации позитивных инструкций

Ты помогаешь строить обучающий датасет для instruction-following retrieval. По паре вопрос-контекст нужно придумать ОДНУ позитивную инструкцию на русском языке. Позитивная инструкция должна:

1. звучать как реальная поисковая инструкция;
2. быть тематически близкой к вопросу;

3. делать данный контекст ПОДХОДЯЩИМ и релевантным;
4. не должна зависеть от скрытой разметки;
5. не должна быть бессмысленной, слишком общей или противоречивой.

Верни JSON с полями: `positive_instruction`, `relevance_reason`.

Затем генерировался текст, который делал `context` релевантным для `query + instruction` (в статье такой текст называют `instruction positive`). И таким же образом 3 текста, которые подходят под `query`, но не подходят под `instruction` (в статье такие называют `instruction negative`).

Промпт для генерации инструктивных негативов

Ты помогаешь строить `instruction-following retrieval` датасет по рецепту `Promptriever`. Для каждого примера нужно создать `passages`, которые позволяют обучать `bi-encoder` учитывать инструкцию на уровне конкретного запроса.

Верни JSON со следующими полями:

1. `generated_positive_passage`: `passage`, релевантный и вопросу, и инструкции
2. `instruction_negative_passages`: список из нескольких `passages`, которые релевантны исходному вопросу, но НЕ удовлетворяют дополнительному условию инструкции

Требования:

1. `passages` должны быть правдоподобными и естественными;
2. нельзя делать `negative` явно абсурдным или нерелевантным теме;
3. `negative` должен отвечать базовому вопросу, но не проходить по `instruction`;
4. верни только JSON без `markdown`.

После этого к каждому `query` искали 2 текста из исходного датасета, которые близки по косинусному расстоянию, но не подходят по смыслу (в статье такие называют `hard negative`). Все эти тексты добавляются в датасет.

Валидация сгенерированных текстов

После генерации текстов, их нужно валидировать, потому что LLM могла сгенерировать текст, который не будет релевантен запросу с инструкцией, хотя должен быть (или наоборот). Для валидации мы используем `bge-reranker-m3-v2` [2]. Каждому тексту и запросу (возможно, с инструкцией) сопоставляет скор – меру сходства. Инструкции и сгенерированные запросы с

низким скором отбрасываются.

Таблица 2: Пороги для сгенерированных текстов / инструкций

Инструкция	Instruction positive текст	Instruction negative текст
≥ 1.0	≥ 3.0	≤ 4.0

К первым 5745 строкам из SberQuAD были сгенерированы позитивные инструкции, 5062 из них были помечены как релевантные. После валидации instruction positive текстов (для каждого запроса генерировалось по 3 текста) 5219 были отмечены как релевантные. После валидации instruction negative 4193 текста были отмечены как релевантные.

Итоговый размер датасета – 1748 строк. В каждой из них было ≥ 1 hard negatives, ≤ 3 instruction negatives и 1 instruction positive текст.

Пример из обучающего датасета

Query: Шедеврами какой музыки признаны сочинения Моцарта?

Instruction: Найдите информацию о музыкальных жанрах, в которых признаны шедевры Моцарта.

Positive: Моцарт - один из наиболее значительных композиторов Венской классической школы, и его сочинения признаны шедеврами в различных музыкальных жанрах, включая симфоническую, концертную, камерную, оперную и хоровую музыку.

Instruction Negative: В музыке Моцарта можно услышать влияние классицизма и барокко...

Hard Negative: Отличительной чертой творчества Моцарта является сочетание строгих, ясных форм...

3.3 Дообучение модели

Дообучение модели производилось с помощью LoRA. Было проведено несколько экспериментов с разными моделями и разными параметрами LoRA. В качестве loss-функции использовалась MultipleNegativesRankingLoss. Каждая модель дообучалась ровно 20 эпох.

Таблица 3: Результаты дообучения с различными параметрами LoRA

Base model	r	α	Dropout	Batch size	Trainable params ¹	nDCG@10	pMRR
m-e5-base	8	16	0	16	0.16% (0.45M)	59.405	−0.08538
m-e5-base	8	16	0.05	16	0.16% (0.45M)	61.392	−0.09782
m-e5-base	16	32	0.05	16	0.32% (0.89M)	59.894	0.61896
m-e5-base	32	64	0.05	16	0.63% (1.75M)	59.785	−0.07843
bge-m3	8	16	0.05	16	0.21% (1.12M)	63.579	−0.08945
bge-m3	16	32	0.05	16	0.41% (2.23M)	61.962	−0.05129

¹ Trainable params – доля обучаемых параметров от всех параметров модели.

После экспериментов с дообучением получили прирост в $> +3$ по метрике pMRR относительно baseline решения, однако также получили заметную просадку в -10 по метрике nDCG@10 (в среднем). Это подтверждает гипотезу, что модель научилась различать некоторые дополнительные инструкции в запросах и менять эмбединг в зависимости от них, однако при этом ее функции ретрива немного ухудшились. На это могло повлиять несколько факторов, как то:

1. размер и качество синтетического датасета; есть риск, что мы плохо отсортировали сгенерированные тексты, и в обучающий датасет попало много шумных примеров, что могло ухудшить качество обучения.
2. параметры LoRA; возможно следовало провести более тщательный подбор гиперпараметров, например, попробовать другой learning rate, большее число эпох, другие значения r и так далее.
3. архитектура модели; возможно, базовая модель не была достаточно мощной для эффективного обучения на инструкциях, и более крупная модель могла бы показать лучший результат.

Лучшая попытка показала прирост в $+3.6$ по метрике pMRR, что является неплохим результатом, учитывая ограниченный размер датасета и относительно слабую модель, использовавшуюся при его генерации.

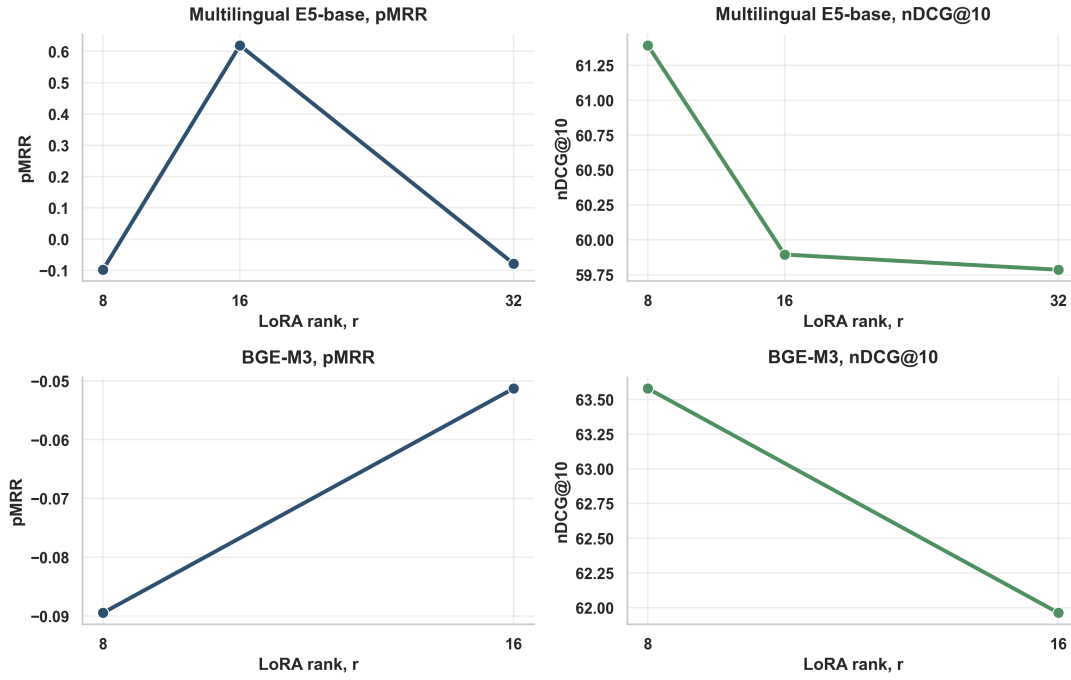


Рис. 1: Зависимость метрик от ранга LoRA

На рисунке 1 показано, как изменение ранга LoRA влияет на качество моделей по метрикам pMRR и nDCG@10.

3.4 Latency

Помимо качества поиска, для retrieval-модели важно время построения эмбединга. Поэтому была добавлена отдельная проверка latency: для одного запроса выполнялось несколько прогонов после warmup, после чего считались среднее значение, медиана и p95 (95%-й перцентиль).

Таблица 4: Latency кодирования одного запроса. Замер проводился на RTX4090.

Модель	Mean, ms	Median, ms	p95, ms
multilingual-e5-base	6.913	6.798	7.409
m-e5-base + LoRA ($r = 16, \alpha = 32$)	6.861	6.805	7.441
bge-m3	12.659	12.552	13.793
bge-m3 + LoRA ($r = 16, \alpha = 32$)	12.448	12.262	14.046

Замер latency проводился следующим образом: для каждой модели выполнялось 200 прогонов кодирования одного запроса с инструкцией, первые 20 прогонов – warmup, а затем для оставшихся 180 прогонов считались среднее значение, медиана и p95.

Можем видеть, что время инференса практически не изменилось между дообученной моделью и базовой. Если сравнивать между собой модели на основе bge-m3 и m-e5-base, то bge-m3 работает примерно в 2 раза медленнее, что связано с ее большим размером (569М параметров против 278М).

4 Заключение

В рамках работы был реализован пайплайн дообучения retrieval-модели на русскоязычных инструкциях по мотивам подхода Promptriever. Был собран датасет на основе SberQuAD: для исходных вопросов генерировались позитивные инструкции, instruction positive тексты, instruction negative тексты и hard negatives. Дополнительно была проведена автоматическая валидация сгенерированных примеров с помощью reranker-модели, чтобы уменьшить количество шумных обучающих пар.

Эксперименты показали, что дообучение с помощью LoRA может заметно улучшить качество следования инструкциям. Наиболее удачный запуск для multilingual-e5-base дал прирост по метрике pMRR относительно baseline при сравнительно небольшой просадке в nDCG@10. Это означает, что модель стала лучше отличать документы, которые просто близки к запросу по теме, от документов, которые действительно удовлетворяют дополнительному условию из инструкции.

При этом результаты остаются ограниченными размером и качеством синтетического датасета. Итоговый набор данных получился сравнительно небольшим, а генерация инструкций и негативов не всегда идеально отражает реальные пользовательские сценарии. Поэтому полученные результаты следует рассматривать как подтверждение применимости подхода.

5 Использование ИИ

Во время работы над проектом использовались ИИ-инструменты для написания, отладки и оптимизации кода. Для этих целей использовался ChatGPT. Все остальное – поиск и анализ литературы, проектирование экспериментов, интерпретация результатов – выполнялось самостоятельно.

6 Дальнейшая работа

В дальнейшем имеет смысл увеличить размер обучающего датасета и провести более строгую фильтрацию синтетических примеров. Сейчас обучение проводится на ограниченной части SberQuAD, поэтому модель видит только часть возможных типов инструкций. Расширение датасета позволит проверить, растёт ли качество instruction following при увеличении числа примеров.

Также важно провести более подробные ablation-эксперименты. В частности, отдельно сравнить вклад hard negatives и instruction negatives, проверить разные значения ранга LoRA, число эпох и learning rate. Это позволит понять, какие компоненты пайплайна действительно дают прирост, а какие почти не влияют на итоговое качество.

Наконец, полезно добавить ручную оценку части сгенерированных инструкций и негативов. Автоматическая валидация через reranker удобна для масштабирования, но она не всегда замечает логические ошибки в инструкции. Ручная проверка небольшой выборки помогла бы точнее оценить качество датасета и объяснить ошибки модели на mFollowIR.

Список использованных источников

- [1] AI@Meta. “Llama 3 Model Card”. В: (2024). URL: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- [2] Jianlv Chen и др. *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2025. arXiv: 2402.03216 [cs.CL]. URL: <https://arxiv.org/abs/2402.03216>.
- [3] Pavel Efimov и др. “SberQuAD – Russian Reading Comprehension Dataset: Description and Analysis”. В: *Experimental IR Meets Multilinguality, Multimodality, and Interaction*. Springer International Publishing, 2020, с. 3–15. DOI: 10.1007/978-3-030-58219-7_1.
- [4] Artem Snegirev и др. *The Russian-focused embedders’ exploration: ruMTEB benchmark and Russian embedding model design*. 2025. arXiv: 2408.12503 [cs.CL]. URL: <https://arxiv.org/abs/2408.12503>.
- [5] Hongjin Su и др. *One Embedder, Any Task: Instruction-Finetuned Text Embeddings*. 2023. arXiv: 2212.09741 [cs.CL]. URL: <https://arxiv.org/abs/2212.09741>.
- [6] Liang Wang и др. “Multilingual E5 Text Embeddings: A Technical Report”. В: *arXiv preprint arXiv:2402.05672* (2024).
- [7] Orion Weller и др. *mFollowIR: a Multilingual Benchmark for Instruction Following in Retrieval*. 2025. arXiv: 2501.19264 [cs.IR]. URL: <https://arxiv.org/abs/2501.19264>.
- [8] Orion Weller и др. “Promptriever: Instruction-Trained Retrievers Can Be Prompted Like Language Models”. В: (2024). arXiv: 2409.11136 [cs.IR]. URL: <https://arxiv.org/abs/2409.11136>.